

Aula 2 — Contando ciclos

A listagem como lupa: pipeline, banco de acesso e pilha

Microcontroladores — DENE/UFMT

OBJETIVOS

- Localizar e ler a listagem gerada pelo compilador, associando cada instrução de máquina à linha de C que a originou.
- Calcular o tempo de execução de um trecho por contagem de ciclos, distinguindo instruções de um ciclo das de dois e do desvio condicional.
- Explicar o sufixo `,c` presente em quase toda instrução gerada, e dizer o que muda no custo quando ele desaparece.
- Construir uma rotina de atraso cuja duração é decidida em tempo de execução, e prever seu período *antes* de medir.
- Identificar as fontes de incerteza dessa previsão e justificar por que contagem de ciclos não substitui temporizador por *hardware*.

1 A duração não se escreve em C

A Aula 1 fechou com uma afirmação que ficou sem prova: a listagem e o mapa de memória são instrumentos de trabalho, não subprodutos descartáveis. Hoje ela é cobrada.

E é cobrada por um problema concreto. Na Oficina 1, vocês produziram notas musicais no *buzzer* comutando um pino dentro de um laço de espera. Para mudar a nota, muda-se a duração da espera — e aí `__delay_ms` não serve, porque ela é uma macro e exige argumento constante em tempo de compilação (Aula 1, §7.6). A duração precisa ser um parâmetro, e não há função de biblioteca que faça isso.

CONCEITO

Quando a duração vira parâmetro, ela deixa de ser propriedade do programa que vocês escreveram e passa a ser propriedade do **código que o compilador gerou**.

São coisas diferentes, e a segunda não está no arquivo `.c`. Uma linha de C não tem duração; um bloco de instruções de máquina tem. Entre uma e outro há uma tradução que ninguém mostrou a vocês ainda.

Esta aula abre essa tradução. Não para escrever em *assembly* — vocês não vão precisar disso no semestre — mas para **ler**, que é uma habilidade muito mais barata e imediatamente útil. Doze mnemônicos bastam.

1.1 O que a lupa vai mostrar

A Aula 1 encerrou com uma seção de contexto — pilha, contador de programa, paralelismo, banco de acesso — apresentada como explicação de fundo, sem uso prático. Hoje cada um desses itens vira um número mensurável, e a listagem é o que os torna visíveis.

	O que a seção estabelece	Onde isso aparece na listagem
2	Como se lê uma listagem	As quatro colunas e a origem em C de cada instrução
3	Uma instrução por ciclo, exceto no desvio	<code>goto</code> , <code>call</code> e <code>return</code> ocupando dois ciclos
4	O banco de acesso é o que torna o C viável aqui	O sufixo <code>,c</code> em quase toda linha gerada
5	Chamar uma função custa, e a pilha não é RAM	Os ciclos de <code>call</code> mais <code>return</code>
6	A construção: atraso ajustável em execução	A contagem de ciclos do laço

7 Por que a previsão erra, e quanto O caminho que só executa às vezes

2 Anatomia de uma listagem

A listagem é um arquivo de texto com extensão `.lst`, produzido pelo montador a partir do que o compilador gerou. No MPLAB X, ela aparece no diretório de construção do projeto; pela linha de comando, o caminho mais direto para inspecionar um trecho é pedir só a tradução:

```
xc8-cc -mcpu=18F4550 -S programa.c -o programa.s
```

Todos os trechos desta aula vêm de uma listagem real, gerada nesta bancada a partir do programa abaixo. Ele não faz nada de útil — foi escrito para produzir poucas instruções e ser lido por inteiro.

```
#include <xc.h>
#include <stdint.h>

volatile uint8_t heater_on = 0;
volatile int16_t temperatura = 250; /* decimos de grau */
volatile int16_t setpoint = 300;

void controla(void)
{
    if (temperatura < setpoint)
        LATCbits.LATC1 = 1, heater_on = 1;
    else
        LATCbits.LATC1 = 0, heater_on = 0;
}
```

2.1 As quatro colunas

```
438                                     ;macro_do_while.c: 87: if (temperatura < setpoint)
439 000802 5001                        movf    _setpoint^0,w,c
440 000804 5C03                        subwf   _temperatura^0,w,c
```

Coluna	Conteúdo
439	Número da linha da listagem. Sem relação com o arquivo <code>.c</code>
000802	Endereço na memória de programa, em bytes. É o valor que o contador de programa terá quando esta instrução for buscada
5001	A instrução propriamente dita: dois bytes gravados na Flash. É isto que vai para o <code>.hex</code>
movf _setpoint^0,w,c	A mesma instrução em forma legível

As linhas iniciadas por ponto e vírgula são comentários que o compilador insere para dizer **de qual linha de C** saiu o bloco seguinte. São elas que transformam a listagem em instrumento de diagnóstico: permitem perguntar “o que esta linha minha virou?” e obter resposta.

NOTA

Repare no endereço: `0x0802` e depois `0x0804`. Instruções ocupam dois bytes e os endereços avançam de dois em dois. É a memória de programa de 16 bits da Aula 1, §4, vista de perto.

2.2 Uma instrução decodificada à mão

Vale fazer isto uma vez na vida, porque desfaz o mistério de forma definitiva. Mais adiante na mesma listagem aparece:

```
453  000820  828B          bsf    139,1,c
```

O manual do dispositivo dá o formato de `BSF` — ligar um bit — como `1000 bbba ffffffff`. Escrevendo `0x828B` em binário:

$$\underbrace{1000}_{\text{código}} \underbrace{001}_b \underbrace{0}_a \underbrace{10001011}_f$$

Campo	Valor	Significado
<i>b</i>	<code>001</code> = 1	O bit a ligar é o bit 1
<i>a</i>	<code>0</code>	Endereço interpretado no banco de acesso
<i>f</i>	<code>10001011</code> <code>0x8B</code>	= Endereço 139 — que é LATC

Ou seja: `LATCbits.LATC1 = 1;` compilou para **uma única instrução de dois bytes**, que executa em 250 ns e liga o bit 1 do endereço `0x8B`.

CONCEITO

Isto é a afirmação central da Aula 1, §3 — “os pinos são posições de memória” — já não como analogia, mas como campo de bits dentro de um opcode. O endereço 139 não é tratado de maneira especial em lugar nenhum: é o mesmo campo *f* que apontaria para uma variável comum.

O que faz `0x8B` acionar um aquecedor é o silício ligado atrás dele, não a instrução.

2.3 O menor programa que aciona um pino

Se `LATCbits.LATC1 = 1` cabe em uma instrução, quanto ocupa um programa completo que acende um LED e para? Este:

```
#include <xc.h>

void main(void)
{
    LATDbits.LATD0 = 1; /* valor no latch */
    TRISDbits.TRISD0 = 0; /* so' entao saida */
    while (1) { }
}
```

Traduzido, é a listagem inteira — três instruções, oito bytes de Flash:

```
000800  808C          bsf    140,0,c      ; LATD bit 0 <- 1
000802  9095          bcf    149,0,c      ; TRISD bit 0 <- 0
000804  EF02 F004      goto   $          ; while (1)
```

Instrução	O que faz, e quando o LED acende
<code>bsf 140,0,c</code>	<i>Bit set file</i> . Liga o bit 0 do endereço 140, que é LATD. Escreve 1 no latch de RD0 — e nada acontece eletricamente . No <i>reset</i> o pino ainda é entrada, em alta impedância. O valor fica guardado, esperando
<code>bcf 149,0,c</code>	<i>Bit clear file</i> . Zera o bit 0 de TRISD; zero é saída. RD0 passa a ser saída e assume o que já estava no latch. O LED acende aqui

`goto $` O `$` é notação do montador para “o endereço desta própria instrução”. Um desvio para si mesma, dois ciclos, indefinidamente. É o `while (1)` do C

CONCEITO

Estas duas primeiras instruções **são** a regra `LAT` antes de `TRIS` da Aula 1, §3, reduzida à sua forma mínima. A regra não é convenção de estilo nem recomendação do fabricante: é a ordem de dois opcodes.

Trocá-las faz o pino virar saída enquanto o latch ainda contém o valor indefinido do *reset*, e só depois receber o valor pretendido. Com um LED, o pulso é invisível. Com o *cooler*, é o tranco que vocês ouviram na Oficina 1.

NOTA

Compare com o exemplo anterior, que gerou 51 instruções. A diferença não está no acionamento do pino — que continua custando uma instrução — e sim em ter duas variáveis globais `int16_t` inicializadas e uma comparação com sinal. O acionamento nunca foi a parte cara.

2.4 Onde está o mapeamento dos pinos

O programa diz `LATDbits.LATD0`; a listagem diz `140`. Entre um e outro há quatro camadas, e apenas a primeira é software.

Camada	Onde vive, e o que pode mudá-la
1 Nome → endereço	No cabeçalho do dispositivo, incluído por <code>xc.h</code> . É uma declaração da forma <code>extern volatile unsigned char LATD __at(0xF8C)</code> ; mais a união de campos de bits que dá <code>LATDbits.LATD0</code> . Pura tabela de nomes — as 216 linhas <code>equ</code> do início da listagem são essa mesma tabela vista pelo montador
2 Endereço → bloco interno	Fixo em silício. Documentado no mapa de registradores da folha de dados. Nenhum ajuste de software altera
3 Bloco interno → pino físico	Fixo em silício. <code>RD0</code> é o pino 19 do encapsulamento de 40 pinos. Está no diagrama de pinagem
4 Pino físico → o que há na placa	Não existe em arquivo nenhum. Está no esquemático da XM118 e na serigrafia — e a serigrafia tem precedência sobre o manual do fabricante

Para inspecionar a camada 1 na sua instalação:

```
grep -n "LATD\b" /opt/microchip/xc8/v2.*/pic/include/proc/pic18f4550.h
```

ATENÇÃO

A camada 4 é a que interessa na bancada, e é a única que nenhuma ferramenta conhece. **Nada no programa diz que `RD0` é um LED** — nem que `RC2` vai ao *cooler* ou ao *buzzer* conforme a posição de `CH3-5` e `CH3-6`.

É exatamente a lacuna descrita na resposta do Exercício 1.6: o compilador não avisa que `LATD = 0x01` desliga o aquecedor porque a informação “o bit 1 deste endereço comanda 12 V” não existe em lugar algum que ele possa ler. Ela mora no esquema elétrico e na cabeça de quem escreveu.

2.5 O código que você não escreveu

Ainda assim, a listagem completa do exemplo maior tem 660 linhas. Vale saber de onde vêm, porque a resposta é útil.

Bloco	Linhas	O que é
-------	--------	---------

Definições equ	216	Um nome para cada registrador do dispositivo, inclusive os do módulo USB que o programa nunca toca. Custa zero de Flash
Diretivas, símbolos, mapa	cerca de 390	Organização do montador e do ligador
Instruções de máquina	51	O programa propriamente dito

E das 51, apenas 33 são suas: 16 pertencem a um bloco de partida que o compilador insere antes de `main`.

```
movlw low __pdataCOMRAM      ; aponta para os valores iniciais, na Flash
movwf tblptrl,c
lfsr 0,__pdataCOMRAM         ; aponta para o destino, na RAM
tblrd *+                     ; le um byte da Flash
movff tablat,postinc0         ; escreve na RAM
bnz copy_data0               ; repete ate' terminar
clrf __pbssCOMRAM            ; zera as nao inicializadas
goto _main
```

CONCEITO

Por que esse bloco existe. A declaração `int16_t temperatura = 250;` promete que a variável vale 250 quando `main` começa. Mas a RAM não guarda nada ao ser energizada: o 250 precisa estar na Flash e ser copiado.

E copiar da Flash não é acesso comum à memória — é a leitura de tabela `TBLRD` anunciada na Aula 1, §4, aqui executando. A separação Harvard, que naquela aula era uma consequência a acreditar, é este laço.

ATENÇÃO

Uma global inicializada custa **três vezes**: bytes de RAM, bytes de Flash com a cópia do valor inicial, e ciclos na partida. Uma global sem inicializador cai apenas no `clrf` que zera a região não inicializada.

Escrever `= 0` numa global não é neutro — é pedir uma cópia da Flash para gravar um zero que já estaria lá.

2.6 Os doze mnemônicos que bastam

O conjunto completo do PIC18 tem **75 instruções** — mais oito se `XINST` for habilitado, o bit de configuração que a Aula 1, §6, manda manter desligado por não ser suportado pelo XC8. Para comparação: o PIC16 tem 35, um Cortex-M0+ tem cerca de 56, e um processador de mesa tem milhares.

CONCEITO

O número engana. O que caracteriza este núcleo não é a quantidade de instruções, e sim três decisões:

Um acumulador só `WREG`. Não há banco de registradores — coerente com a afirmação da Aula 1, §3, de que toda posição de memória é um registrador.

Operação direta sobre a memória `bsf 140,0` altera a memória sem carregar nada antes.

Largura fixa toda instrução ocupa uma palavra, exceto `goto`, `call`, `lfsr` e `movff`, que ocupam duas.

A segunda é o oposto de uma arquitetura *load-store*, em que a memória só é tocada por instruções de carga e armazenamento. Ligar um bit de porta custa uma instrução aqui e três num Cortex-M — que ainda assim termina antes, por operar a uma frequência muito maior. Contagem de instruções não é medida de desempenho.

Praticamente tudo que o XC8 gera em código de controle cabe nesta lista.

Instrução	Efeito	Observação
<code>movlw k</code>	$W \leftarrow k$	Carrega uma constante no acumulador
<code>movwf f</code>	$f \leftarrow W$	Escreve o acumulador na memória
<code>movf f, w</code>	$W \leftarrow f$	Lê da memória; afeta o indicador de zero
<code>clrf f</code>	$f \leftarrow 0$	Zera uma posição
<code>bsf f, b</code>	Liga o bit b de f	Uma instrução, sem ler-modificar-escrever visível no código
<code>bcf f, b</code>	Desliga o bit b de f	—
<code>btfsc f, b</code>	Pula a próxima instrução se o bit for 0	Custo variável; vide §3
<code>btfss f, b</code>	Pula a próxima instrução se o bit for 1	—
<code>subwf f, w</code>	$W \leftarrow f - W$	Comparações saem daqui
<code>subwfb f, w</code>	Subtração com empréstimo	Byte alto de operações de 16 bits
<code>goto k</code>	Desvio incondicional	Duas palavras, dois ciclos
<code>call k</code>	Chamada de função	Empilha o retorno
<code>return</code>	Retorno	Desempilha
<code>xorlw k</code>	$W \leftarrow W \oplus k$	Aparece na comparação com sinal; vide §3

O sufixo `,w` significa que o resultado vai para o acumulador em vez de voltar à memória. O sufixo `,c` é assunto da seção 4. O `^0` que acompanha os nomes é notação do montador para o deslocamento dentro do banco — pode ser ignorado na leitura.

3 Um ciclo por instrução, exceto quando não

A Aula 1, §8.3, afirmou que programa e dados em barramentos separados permitem buscar a próxima instrução enquanto a atual executa, e que por isso o processador completa uma instrução por ciclo. Também afirmou que desvios são exceção. Agora dá para usar isso.

CONCEITO

Por que o desvio custa o dobro. Enquanto a instrução no endereço n executa, a do endereço $n + 2$ já está sendo buscada. Se a instrução em n for um desvio, a próxima a executar não é a de $n + 2$: a busca adiantada foi trabalho perdido, e o processador gasta um ciclo adicional buscando o destino verdadeiro.

Não é penalidade nem defeito. É o preço de um mecanismo que, no resto do tempo, entrega uma instrução por ciclo de graça.

Instrução	Ciclos	Observação
Aritmética, lógica, movimentação, bit	1	A maioria esmagadora do código
<code>goto</code> , <code>call</code> , <code>return</code> , <code>bra</code>	2	Sempre; o desvio é sempre tomado

<code>btfsc</code> / <code>btfss</code> que não pula	1	Segue em frente; nada foi descartado
<code>btfsc</code> / <code>btfss</code> que pula uma instrução de uma palavra	2	Descarta a busca adiantada
<code>btfsc</code> / <code>btfss</code> que pula uma instrução de duas palavras	3	Precisa descartar as duas palavras

ATENÇÃO

A última linha é a mais esquecida, e `goto` é justamente uma instrução de duas palavras. Um desvio condicional que pula um `goto` custa três ciclos. Como o XC8 sem otimização gera exatamente esse padrão o tempo todo, o erro se acumula rápido em qualquer contagem manual.

3.1 Contando uma função inteira

A listagem completa de `controla()`, com os ciclos anotados. `C` é o indicador de transporte, que a comparação deixou preparado.

```

    _controla:
000802 movf  _setpoint^0,w,c      1
000804 subwf _temperatura^0,w,c   1
000806 movf  (_temperatura+1)^0,w,c 1
000808 xorlw 128                  1
00080A movwf ??_controla^0,c      1
00080C movf  (_setpoint+1)^0,w,c   1
00080E xorlw 128                  1
000810 subwfb ??_controla^0,w,c    1
000812 btfsc status,0,c           1 ou 3
000814 goto  u11                  2 (so' se C = 1)
000818 goto  u10                  2 (so' se C = 0)
00081C u11: goto l17               2
000820 u10: bsf 139,1,c            1
000822 movlw 1                    1
000824 movwf _heater_on^0,c        1
000826 goto  l19                  2
00082A l17: bcf 139,1,c            1
00082C clrf  _heater_on^0,c        1
00082E l19: return                2

```

Trecho	Aquecedor liga	Aquecedor desliga
Comparação de 16 bits (8 instruções)	8	8
<code>btfsc</code>	3 (pula)	1 (não pula)
Desvios até o ramo	2	4
Corpo do ramo	3	2
Saída do ramo	2	—
<code>return</code>	2	2
<code>call</code> , no chamador	2	2
Total	22 ciclos	19 ciclos

A 250 ns por ciclo: 5,5 μ s num caso, 4,75 μ s no outro.

CONCEITO

A mesma função, dois tempos. Nenhuma linha do programa mudou entre uma coluna e outra — mudou o valor da temperatura. O tempo de execução deste código depende do dado, e a diferença é de 750 ns.

Isso tem nome: **jitter**, a variação do instante em que algo acontece. Num LED é irrelevante. Numa nota musical é desafinação. Num controle amostrado, é ruído introduzido pelo próprio programa. A seção 7 volta a isso com número.

NOTA

Os dois `xorlw 128` são a comparação com sinal: inverter o bit mais significativo transforma a comparação de números com sinal numa comparação sem sinal, que é o que o indicador de transporte sabe fazer. Oito instruções para comparar dois `int16_t` — o preço concreto da afirmação da Aula 1, §7.4, de que cada operação de 16 bits vira duas ou três instruções.

4 O sufixo `,c`: o banco de acesso

Quase toda instrução da listagem termina em `,c`. Não é decoração.

A Aula 1, §8.4, explicou que os 2 048 bytes de RAM são divididos em bancos de 256 porque o campo de endereço da instrução tem apenas 8 bits — e que, para não pagar troca de banco a cada acesso, o PIC18 mantém uma janela de 256 bytes reunindo os 96 primeiros bytes de RAM aos registradores de periférico do topo do mapa. O bit *a* do opcode é quem escolhe entre a janela e o banco corrente.

Bit <i>a</i>	Na listagem	O que acontece
0	<code>,c</code>	Endereço resolvido na janela de acesso. Uma instrução, um ciclo
1	(ausente)	Endereço resolvido no banco apontado por <code>BSR</code> . Se o banco não for o corrente, o compilador precisa emitir antes uma instrução <code>movlb</code> — mais uma palavra de Flash e mais um ciclo

No programa de exemplo, todas as variáveis globais couberam na janela:

```
_setpoint    0x0001    _heater_on    0x0005
_temperatura 0x0003    ??_controla 0x0006
```

ATENÇÃO

A janela tem 96 bytes de RAM. Um projeto que ultrapasse isso em variáveis frequentemente acessadas começa a ter parte delas fora do banco de acesso — e o código que as usa fica mais lento **sem que nenhuma linha desse código tenha mudado**.

É o primeiro caso do semestre em que acrescentar uma variável em um módulo altera a temporização de outro. Quando uma rotina calibrada começar a errar depois de o projeto crescer, esta é a hipótese.

CONTEXTO

Somando ao endereçamento indireto e ao multiplicador por *hardware*, é o banco de acesso que sustenta a afirmação de que o PIC18 foi projetado para receber código de compilador C. Sem ele, cada acesso a variável exigiria gerenciar `BSR` explicitamente, e o código gerado seria perto do dobro.

5 Chamar uma função custa, e a pilha não é RAM

Na contagem da seção 3, quatro dos 22 ciclos foram gastos em `call` e `return` — cerca de 18% do total, sem que nenhum trabalho útil fosse feito.

CONCEITO

Onde vai o endereço de retorno. Numa arquitetura convencional, o `call` empilha o endereço de retorno na mesma RAM onde vivem as variáveis. Aqui não: a pilha de retorno é um bloco de silício dedicado, com 31 níveis, fora do espaço de dados (Aula 1, §8.2).

A consequência prática é agradável: **chamadas aninhadas não consomem os 2 KB de RAM**. A desagradável é que a profundidade é fixa em 31 e não há como aumentá-la — o que só é problema com recursão.

5.1 Macro em vez de função

Se a chamada custa quatro ciclos e a função é curta, a alternativa é a macro: o corpo é expandido no lugar da chamada, e os quatro ciclos desaparecem.

ATENÇÃO

A troca não é gratuita. Cada uso da macro duplica o código na Flash. Com 32 KB de Flash contra 2 KB de RAM, a troca costuma valer a pena — mas é uma escolha de orçamento, não uma otimização automática.

E não conte com `inline`: a licença gratuita do XC8 compila sem otimizar, e a listagem mostra as chamadas intactas.

Uma macro de várias instruções precisa de uma casca sintática, e a casca usual é `do { ... } while(0)`:

```
#define LIGA_HEATER() do { LATCbits.LATC1 = 1; heater_on = 1; } while(0)
```

CONCEITO

Por que essa forma, e não outra. O objetivo é que a macro se comporte como uma instrução simples, sobrevivendo a um `if` sem chaves. Há três candidatas, e o que as separa é o comportamento quando o usuário escreve — ou esquece — o ponto e vírgula final.

Definição	Uso com ;	Uso sem ;
<code>do { ... } while(0)</code>	correto	erro de sintaxe
<code>if(1) { ... }</code>	erro de sintaxe	compila e faz o errado
<code>{ ... }</code>	erro de sintaxe	correto

A casa perigosa é a do meio: sem o ponto e vírgula, um `else` escrito pelo usuário se liga ao `if(1)` interno da macro, e o ramo `else` vira código morto. Compila limpo e o defeito só aparece em execução.

A forma `do/while(0)` é a única que funciona com o ponto e vírgula natural e cuja omissão produz erro de compilação. Não se ganha nada em tamanho de código: a condição é constante e o laço desaparece na tradução — as duas formas geram exatamente as mesmas instruções. Ganha-se a impossibilidade de falhar em silêncio.

NOTA

Dentro de `do/while(0)` é lícito usar `break` como saída antecipada, o que permite escrever macros de guarda:

```
#define ATUALIZA(v) do { \
    if ((v) > LIMITE_SEG) break; \
    LATD = (v); \
} while(0)
```

Para macros de uma expressão só — `#define LIGA(r,b) ((r) |= (1U << (b)))` — nada disso se aplica: use parênteses e mantenha a forma de expressão.

6 Construção: um atraso em unidades de iteração

Agora há material suficiente para construir a rotina que motivou a aula.

```
void atraso_unidades(uint16_t unidades)
{
    while (unidades > 0u) {
        unidades--;
    }
}
```

Nada nesse código expressa duração. A duração é uma propriedade do que ele virou, e a listagem informa quanto custa uma volta do laço.

EXPERIMENTO

Procedimento — e é ele que vocês repetem no Roteiro 2.

1. Compilar e abrir a listagem. Localizar o comentário com o número da linha do `while` e ler o bloco até o `goto` que volta ao início.
2. Anotar cada instrução com seu custo, pela tabela da seção 3. Atenção aos desvios condicionais que pulam `goto`.
3. Somar. O resultado é b , o custo de uma iteração, em ciclos.
4. Somar separadamente o que executa uma única vez: `call`, preparação do parâmetro, teste final, `return`. O resultado é a , a sobrecarga.

CONCEITO

O modelo. Chamando a rotina com N unidades, o tempo total é

$$t(N) = (a + b \cdot N) \cdot T_{cy}$$

com $T_{cy} = 250$ ns. É uma reta: b é a inclinação, a o intercepto.

Duas grandezas, duas incógnitas — e por isso **duas** medições de frequência com valores diferentes de N determinam ambas experimentalmente, sem abrir a listagem. É assim que a previsão da contagem pode ser confrontada com a bancada.

Usada para gerar um sinal quadrado, a rotina ocupa cada meio período:

```
while (1) {
    LATDbits.LATD0 ^= 1;
    atraso_unidades(N);
}
```

$$T = 2 \cdot (a + b \cdot N + c) \cdot T_{cy}$$

onde c é o custo da inversão do pino e do laço externo — que, na prática, se soma a a e não é separável por medida de frequência. Os dois viram um intercepto só.

NOTA

Uma constante de compilação vale mais que um número solto no código. A ligação entre unidades e microssegundos é a razão $b \cdot T_{cy}$, e ela merece nome:

```
#define DECIMOS_US_POR_ITER 23 /* medido nesta bancada */
```

Guardar em décimos de microssegundo evita ponto flutuante, pela mesma razão que as temperaturas são guardadas em décimos de grau (Aula 1, §7.4).

7 Onde a previsão erra, e quanto

Uma contagem de ciclos honesta erra. Vale saber por onde, antes de a bancada mostrar.

7.1 O caminho que só executa às vezes

O contador é `uint16_t` e o núcleo é de 8 bits. Decrementar exige tratar o byte alto — mas o byte alto só muda quando o byte baixo passa por zero, uma vez a cada 256 iterações. O compilador gera esse tratamento como um desvio condicional, e o custo da iteração depende de qual caminho foi tomado.

ATENÇÃO

A consequência é que b não é um inteiro. É uma média, e o valor instantâneo oscila. Para um atraso longo isso se dilui; para gerar uma nota musical, é desafinação de fração de semitom.

Um contador `uint8_t` elimina essa variação — ao custo de limitar N a 255. A escolha do tipo, aqui, é uma escolha de forma de onda.

7.2 As outras fontes

Fonte	Ordem	Comentário
Byte alto do contador	fração de ciclo	Diluída em N grande; visível em N pequeno
Quantização da constante	até 2%	23 décimos representa 2,3 μs onde o real pode ser 2,26
Tolerância do cristal	0,001% a 0,01%	Desprezível aqui; decisiva na comunicação serial
Interrupções	sem limite	Um tratador executando durante o atraso o estende pelo tempo que levar. Aula 9

CONCEITO

O limite do método. Contagem de ciclos entrega precisão da ordem de 1% e **nenhuma garantia**: qualquer coisa que interrompa o laço estraga a conta, e o processador fica integralmente ocupado durante a espera.

É a mesma objeção que a Aula 1, §7.6, levantou contra `__delay_ms`, agora com número. A saída não é contar melhor — é deixar de contar: um temporizador por *hardware* conta em paralelo, sem consumir instruções e sem ser afetado pelo que o programa faz. É o assunto do encontro 7, e a razão de ele existir.

DIVERGÊNCIA

A rotina construída hoje não é descartada quando o temporizador chegar. Ela permanece útil exatamente onde o temporizador é caro demais: esperas de poucos microssegundos em protocolos de sensor, em que o

custo de armar e desarmar um temporizador excede a espera. Saber contar ciclos continua valendo; o que muda é a escala em que se aplica.

8 Transposição

O instrumento existe em qualquer plataforma, com outro nome.

Ambiente	Como obter a tradução
XC8 / PIC18	<code>xc8-cc -S</code> , ou o arquivo <code>.lst</code> do diretório de construção
GCC, qualquer alvo	<code>gcc -S</code> , ou <code>objdump -d</code> sobre o binário já ligado
ARM Cortex-M	<code>arm-none-eabi-objdump -d</code> ; no STM32CubeIDE, a janela de desmontagem

CONTEXTO

O que **não** se transpõe é a contagem. No PIC18, uma instrução simples custa um ciclo, sempre, e a tabela da seção 3 cabe em cinco linhas. Num Cortex-M típico, o mesmo trecho pode custar diferente conforme o alinhamento na Flash, o número de estados de espera da memória naquela frequência, e a presença de um acelerador de leitura com comportamento próximo ao de uma memória associativa.

Contar ciclos deixa de ser confiável, e é por isso que aquelas plataformas oferecem um contador de ciclos em *hardware* para medir o que já não se consegue prever. Previsibilidade é uma propriedade que se perde ao subir de porte — a mesma discussão de determinismo do Exercício 1.9, agora do lado do código gerado. Ela reaparece no seminário comparativo.

9 Antes de descer para o laboratório

O Roteiro 2 mede o que esta aula previu. Preenchem a tabela agora, com a teoria fresca, e levem rubricada.

NOTA

Vale o mesmo da Aula 1: previsão errada não é problema; previsão feita depois do experimento é.

Pergunta	Previsão e justificativa	Rubrica
P1. Quantos ciclos custa uma iteração do laço de <code>atraso_unidades</code> ? Conte pela listagem antes de medir		
P2. Com $N = 400$, que frequência você espera no pino?		
P3. Dobrando N para 800, a frequência cai exatamente pela metade? Justifique em termos de a e b		
P4. Trocando o contador para <code>uint8_t</code> e mantendo $N = 200$, o que acontece com a forma de onda?		

P5. O que aconteceria com a medida se houvesse uma interrupção periódica ativa?

BANCADA

Leve para a bancada: esta folha preenchida, a listagem impressa ou aberta, e a contagem de ciclos já feita. Medir sem previsão transforma o roteiro em digitação.

10 Exercícios

TAREFA

Exercício 2.1. Na contagem da seção 3, o caminho “aquecedor liga” custou 22 ciclos e o caminho “aquecedor desliga”, 19.

- (a) Explique a origem exata dos três ciclos de diferença.
- (b) Essa função é chamada a cada 10 ms num laço de controle. Qual é o erro relativo introduzido no período de amostragem?
- (c) Suponha agora que ela seja chamada dentro do laço que gera a nota musical do *buzzer*, a 550 Hz. Qual o efeito audível?

TAREFA

Exercício 2.2. Duas medições feitas nesta bancada, com o programa da seção 6 gerando onda quadrada em RD0:

N	Frequência medida
400	551,0 Hz
800	276,2 Hz

- (a) Determine b (ciclos por iteração) e a (sobrecarga, em ciclos).
- (b) Compare b com o que você contou na listagem. Se divergirem, aponte candidatos.
- (c) Que frequência o modelo prevê para $N = 1200$?
- (d) Por que duas medições bastam, e o que uma terceira acrescentaria?

TAREFA

Exercício 2.3. Um estudante troca `uint16_t` por `uint32_t` no parâmetro de `atraso_unidades`, “para poder esperar mais”.

- (a) O que acontece com b ?
- (b) O atraso máximo obtível aumenta na proporção esperada?
- (c) Proponha uma solução melhor para atrasos longos, sem alterar o tipo.

TAREFA

Exercício 2.4. Um projeto cresce e passa a declarar 140 bytes de variáveis globais. Uma rotina de temporização que estava calibrada começa a produzir períodos cerca de 10% maiores, sem que uma única linha dela tenha sido alterada.

Explique o mecanismo e proponha duas providências.

TAREFA

Exercício 2.5. Considere as duas definições:

```
#define LIGA_A() do { LATCbits.LATC1 = 1; heater_on = 1; } while(0)
#define LIGA_B() if(1){ LATCbits.LATC1 = 1; heater_on = 1; }
```

- (a) Compare o código de máquina gerado pelas duas.
- (b) Exiba um trecho de programa em que uma funciona e a outra não, e diga qual erro o compilador emite em cada caso.
- (c) Um colega argumenta que LIGA_B é preferível por ser mais legível. Responda.